

Concise Report

Company Name:

Website Link:

Date:

Time:

http://localhost:4000

2024-10-15

15:42:15

Risk Summary

High Risk Vulnerabilities

Vulnerability Name	Count	Count %	Description	Mitigation
Open Redirect	1	(5.9%)	The description of the vulnerability is that open redirects are one of the OWASP 2010 Top Ten vulnerabilities. Open redirects occur when an application allows user-supplied input to control an offsite redirect, which can be exploited by spammers or phishing attacks. This is generally a pretty accurate way to find where 301 or 302 redirects could be exploited. For example, an attacker could supply a user with a link containing user input in the location header. The vulnerability can be identified by looking at user-supplied input in query string parameters and POST data to identify where open redirects might be possible.	Assume all input is malicious. Reject any input that does not strictly conform to specifications. Use a list of approved URLs or domains to be used for redirection. Use an intermediate disclaimer page that provides the user with a clear warning that they are leaving the current site.

Medium Risk Vulnerabilities

Vulnerability Name	Count	Count %	Description	Mitigation
CSP: Wildcard Directive	16	(94.1%)	Description of the vulnerability: The vulnerability is related to the handling of Content Security Policy (CSP) in an application. It allows an attacker to bypass the security restrictions imposed by the CSP and potentially execute arbitrary code on the target system. The vulnerability can be summarized as follows: 1.'script-src unsafe-eval 10055-11 CSP: Meta Policy Invalid Directive 10055-12 CSP: Header & Meta 10055-2 CSP: X-WebKit-CSP 10055-3 CSP: Notices 10055-4 CSP: Wildcard Directive 10055-5 CSP: script-src unsafe-inline 10055-6 CSP: style-src unsafe-inline 10055-7 CSP: script-src unsafe-hashes 10055-8 CSP: style-src unsafe-hashes 10055-9 CSP: Malformed Policy (Non-ASCII) Code org/zaproxy/zap/extension/pscanrules/ContentSecurityPolicyScanRule.java: An unsafe script is allowed to evaluate user input, which can lead to arbitrary code execution. 2. 'X-Content-Security-Policy 10055-10 CSP: script-src unsafe-eval 10055-1 CSP: Meta Policy Invalid Directive 10055-13 CSP: Header & Meta 10055-4 CSP: Wildcard Directive	X-Frame- options allows developers of web content to restrict the usage of their application within the form of overlays, frames, or iFrames. The developer can indicate from which domains can frame the content. Implementation of this protection mechanism is in development to cover gaps. There is a need for allowing frames from multiple domains. improper usage of frames in web applications.

Content Security Policy (CSP) Header Not Set	26	(152.9%)	The vulnerability can be described as a missing Content Security Policy (CSP) header and an obsolete Content Security Policy (CSP) header being found in the system. The first vulnerability (10038-1) refers to a missing Content Security Policy (CSP) header, while the second vulnerability (10038-2) refers to an obsolete Content Security Policy (CSP) header. The third vulnerability (10038-3) indicates the presence of a report-only header within the Content Security Policy (CSP). To address this vulnerability, it is crucial to set the Content Security Policy (CSP) headers correctly and ensure that they are up-to-date and relevant to the application's requirements. This can be achieved by reviewing and modifying the existing Content Security Policy (CSP) settings or by implementing a new one if necessary. Additionally, removing any report-only headers from the Content Security Policy (CSP) is essential to mitigate the third vulnerability.	Use vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. Run the code in a "jail" or similar sandbox environment that enforces strict boundaries between the process and the operating system. If possible, create isolated accounts with limited privileges. Assume all input is malicious.
Missing Anti-clickjacking Header	25	(147.1%)	The vulnerability can be described as a non-compliant configuration of the X-Frame-Options header in an application, which may lead to clickjacking attacks. The Anti-clickjacking header is missing, and there are multiple entries for the X-Frame-Options header, some of which have malformed code. This can potentially allow an attacker to bypass the clickjacking protection provided by the header.	Compartmentalize the system to have "safe" areas where trust boundaries can be unambiguously drawn. Architects and designers should rely on the principle of least privilege to decide the appropriate time to use privileges and the time to drop privileges. Always be careful when interfacing with a compartment outside of the safe area.
Vulnerable JS Library	2	(11.8%)	The vulnerability in ExampleLibrary version x.y.z can be described as a security issue where the library does not properly handle certain types of input, potentially leading to arbitrary code execution. This can be exploited by malicious actors to compromise the security of systems utilizing this library. The solution to this vulnerability is to upgrade to the latest version of ExampleLibrary, which has implemented proper handling of input and addresses this security issue.	Set the SameSite attribute of a sensitive cookie to 'Lax' or 'Strict' This instructs the browser to apply this cookie only to same-domain requests. This provides a good Defense in Depth against CSRF attacks. While this mitigation is effective for protecting cookies from a browser's own scripting engine, third-party components or plugins may have their own engines that allow access to cookies.

Low Risk Vulnerabilities

Vulnerability Name	Count	Count %	Description	Mitigation
Application Error Disclosure	1	(5.9%)	The vulnerability is that an error/warning message on the page may reveal sensitive information such as the location of the file that produced the unhandled exception. This information can be used to launch further attacks against the web application. The alert could be a false positive if the error message is found inside a documentation page.	Perform thorough input validation (i.e.: server side validation) on the cookie data if you're going to use it for a security-related decision. Add integrity checks to detect tampering. Protect critical cookies from replay attacks, since cross-site scripting or other attacks may allow attackers to steal a strongly-encrypted cookie.

Cookie without SameSite Attribute	4	(23.5%)	The vulnerability can be described as follows: Affected Software: ZAP Proxy with CookieSameSiteScanRule enabled Vulnerability Overview: The CookieSameSiteScanRule scans for cookies that lack the SameSite attribute and cookies that have an invalid SameSite attribute. The lack of the SameSite attribute can lead to cross-site scripting (XSS) attacks, while an invalid SameSite attribute can result in arbitrary code execution. Description of the Vulnerabilities: 1. Vulnerability 10054-1: Cookie without SameSite Attribute This vulnerability occurs when the CookieSameSiteScanRule scans for cookies that do not have the SameSite attribute. Attackers can exploit this vulnerability by setting cookies without the SameSite attribute, which can lead to cross-site scripting (XSS) attacks. 2. Vulnerability 10054-2: Cookie with SameSite Attribute This vulnerability occurs when the CookieSameSiteScanRule scans for cookies that have the SameSite attribute. There is no known vulnerability associated with this scenario. 3. Vulnerability 10054-3: Cookie with Invalid SameSite Attribute This vulnerability occurs when the CookieSameSiteScanRule scans for cookies that have an invalid SameSite attribute. Attackers can exploit this vulnerability by setting cookies with an invalid SameSite attribute, which can result in arbitrary code execution. Mitigation: Disable the CookieSame	Consider using language-theoretic security (LangSec) techniques that characterize inputs using a formal language. Use an input validation framework such as Struts or the OWASP ESAPI Validation API. Explicitly set the encoding you are using whenever the protocol allows you to.
Cross-Domain JavaScript Source File Inclusion	25	(147.1%)	The description of the vulnerability is: "A Cross-Site Scripting (XSS) vulnerability exists in the page where one or more script files from a third-party domain are included. This vulnerability can be exploited by malicious actors to inject malicious scripts into web pages viewed by other users."	
Server Leaks Information via "X-Powered-By" HTTP Response Header Field	63	(320.9%)	The vulnerability can be described as follows: The web/application server is leaking information via one or more "X-Powered-By" HTTP response headers. This can potentially allow attackers to identify other frameworks/components that the web application relies upon and the vulnerabilities of such components.	
Timestamp Disclosure - Unix	1	(5.9%)	The vulnerability is related to the disclosure of timestamps by the application/web server in Unix systems. The disclosure of timestamps can potentially lead to sensitive information being exposed, as timestamps often contain date and time information that can be used to identify specific events or users. However, the provided information does not specify whether the disclosed timestamps are sensitive or not. To assess the severity of this vulnerability, one would need to determine whether the disclosed timestamps are sensitive or not. If they are sensitive, then the vulnerability can be classified as a high or critical severity vulnerability, depending on the impact it can have on the system and its users. If they are not sensitive, then the vulnerability can be classified as a low or informational severity vulnerability, depending on its impact on the system and its users. Code: org/zaproxy/zap/extension/pscanrules/TimestampDisclosureScanRule.java This rule scans for the disclosure of timestamps by the application/web server in Unix systems. If a timestamp is disclosed, the rule will trigger an alert. However, the provided information does not specify whether the disclosed timestamps are sensitive or not. Therefore, it is important to manually confirm that the disclosed timestamps are not sensitive before relying on this rule. In summary, this vulnerability is related to the disclosure of timestamps by the application/web server in Unix systems. The severity	

X-Content-Type-Options Header Missing	40	(235.3%)	The vulnerability allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type.	No Mitigation Available
---------------------------------------	----	----------	--	-------------------------